

## Contents of Demo Flow

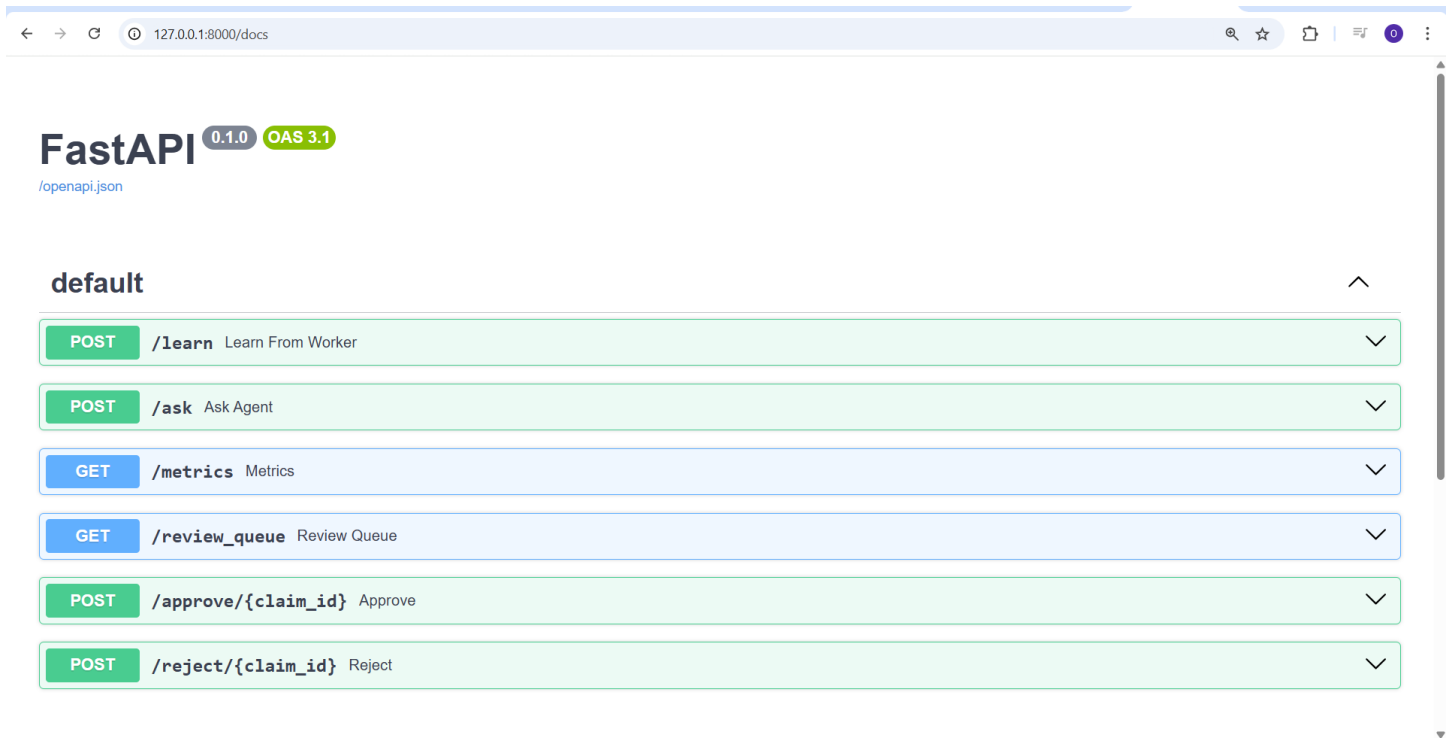
|                                                    |   |
|----------------------------------------------------|---|
| Step 1 — Start System.....                         | 2 |
| Step 2 — Knowledge Acquisition Demo .....          | 2 |
| Step 3 — Conflict Detection Demo.....              | 3 |
| Step 4 — Human Review Queue .....                  | 5 |
| Step 5 — Retrieval Demo .....                      | 5 |
| Step 6 — Human Approval & Memory Replacement ..... | 6 |
| Step 6A — Ask Before Approval .....                | 6 |
| Step 6B — Human Approval .....                     | 7 |
| Step 6C — Ask Again After Approval .....           | 8 |
| Step 7 — Metrics Demo .....                        | 9 |

### Link video demo:

[https://drive.google.com/file/d/1aG1McRpAZThrCny\\_Dxfc43QOWU34LP9g/view?usp=sharing](https://drive.google.com/file/d/1aG1McRpAZThrCny_Dxfc43QOWU34LP9g/view?usp=sharing)

## Step 1 — Start System

- Terminal: run  
“ollama run tinyllama”  
“uvicorn app.main:app --reload”
- Browser: <http://127.0.0.1:8000/docs>



- Explain:
  - + System runs fully local using FastAPI + Ollama.
  - + This simulates an edge-deployable industrial AI assistant.

## Step 2 — Knowledge Acquisition Demo

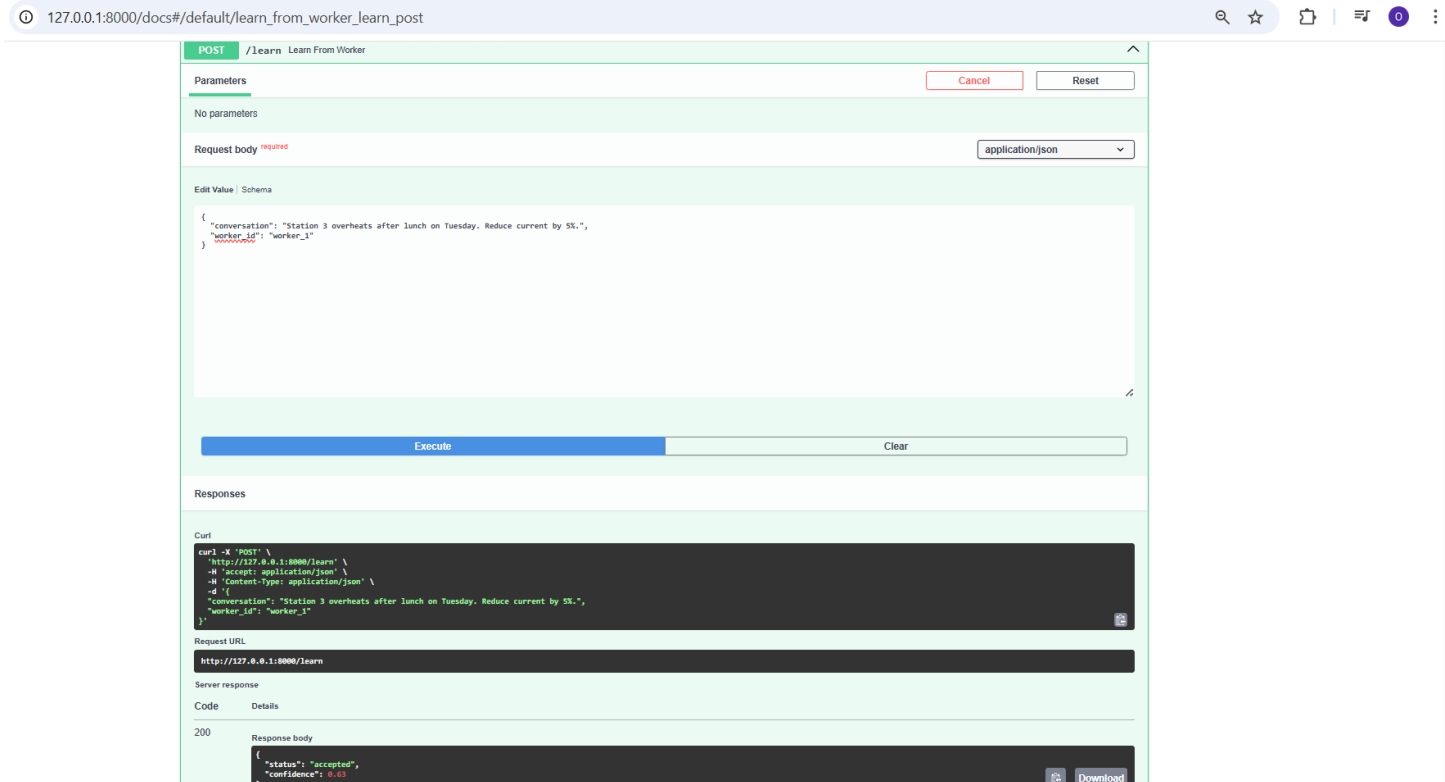
API: **POST /learn**

- Input:

```
{
  "conversation": "Station 3 overheats after lunch on Tuesday. Reduce current by 5%.",
  "worker_id": "worker_1"
}
```

- Output:
 

```
{
  "status": "accepted",
  "confidence": 0.63
}
```



- Explain: The system converts worker conversations into structured operational knowledge instead of storing raw chat logs.

### Step 3 — Conflict Detection Demo

- Input:
 

```
{
  "conversation": "Station 3 overheats after lunch on Tuesday. Increase current by 5%",
  "worker_id": "worker_2"
}
```
- Output:
 

```
{
  "status": "conflict",
  "confidence": 0.2
}
```

127.0.0.1:8000/docs#/default/learn\_from\_worker\_learn\_post

Request body required application/json

Edit Value | Schema

```
{
  "conversation": "Station 3 overheats after lunch on Tuesday. Increase current by 5%.",
  "worker_id": "worker_2"
}
```

Execute Clear

Responses

Curl

```
curl -X 'POST' \
  'http://127.0.0.1:8000/learn' \
  -H 'accept: application/json' \
  -H 'content-type: application/json' \
  -d '{
    "conversation": "Station 3 overheats after lunch on Tuesday. Increase current by 5%.",
    "worker_id": "worker_2"
  }'
```

Request URL

```
http://127.0.0.1:8000/learn
```

Server response

| Code | Details                                                                                                 |
|------|---------------------------------------------------------------------------------------------------------|
| 200  | <p>Response body</p> <pre>{   "status": "conflict",   "confidence": 0.2 }</pre> <p>Response headers</p> |

- Explain: The system detects contradictory operational knowledge and prevents blind self-learning.  
→ anti-poisoning architecture

## Step 4 — Human Review Queue

### API: GET /review\_queue

127.0.0.1:8000/docs#/default/review\_queue\_review\_queue\_get

GET /review\_queue Review Queue

Parameters

No parameters

Execute Clear

Responses

Curl

```
curl -X 'GET' \
  'http://127.0.0.1:8000/review_queue' \
  -H 'accept: application/json'
```

Request URL

http://127.0.0.1:8000/review\_queue

Server response

Code Details

200

Response body

```
{
  "claim_id": "fdee5b8a-e410-47b5-80b5-f1e8e397cd1",
  "entity": "station 3",
  "condition": "after lunch on Tuesday",
  "action": "increase current by 5%",
  "outcome": "prevent overheating",
  "confidence": 0.2,
  "source_worker": "worker_2",
  "status": "conflict",
  "timestamp": "2026-05-13 19:24:06.797497"
}
```

Response headers

```
content-length: 282
content-type: application/json
date: Wed, 13 May 2026 12:25:35 GMT
server: unicorn
```

Responses

| Code | Description | Links |
|------|-------------|-------|
|------|-------------|-------|

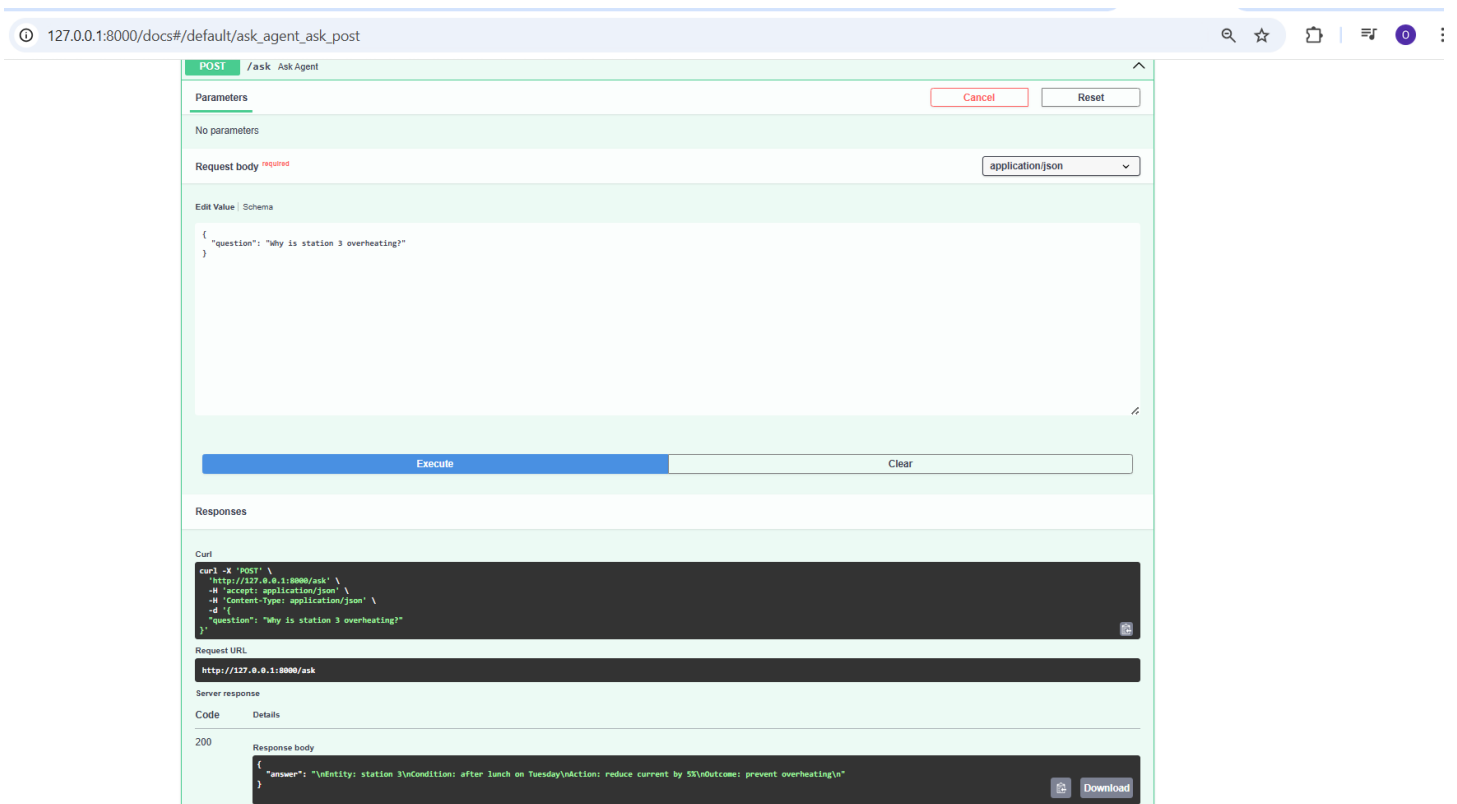
- Explain: Conflicting claims are quarantined and escalated to human supervisors instead of automatically trusted.

## Step 5 — Retrieval Demo

### API: POST /ask

- Input:

```
{
  "question": "Why is station 3 overheating?"
}
```
- Output:  
Entity: station 3  
Condition: after lunch on Tuesday  
Action: reduce current by 5%  
Outcome: prevent overheating



- Explain: Previously learned operational knowledge is reused in future worker interactions.

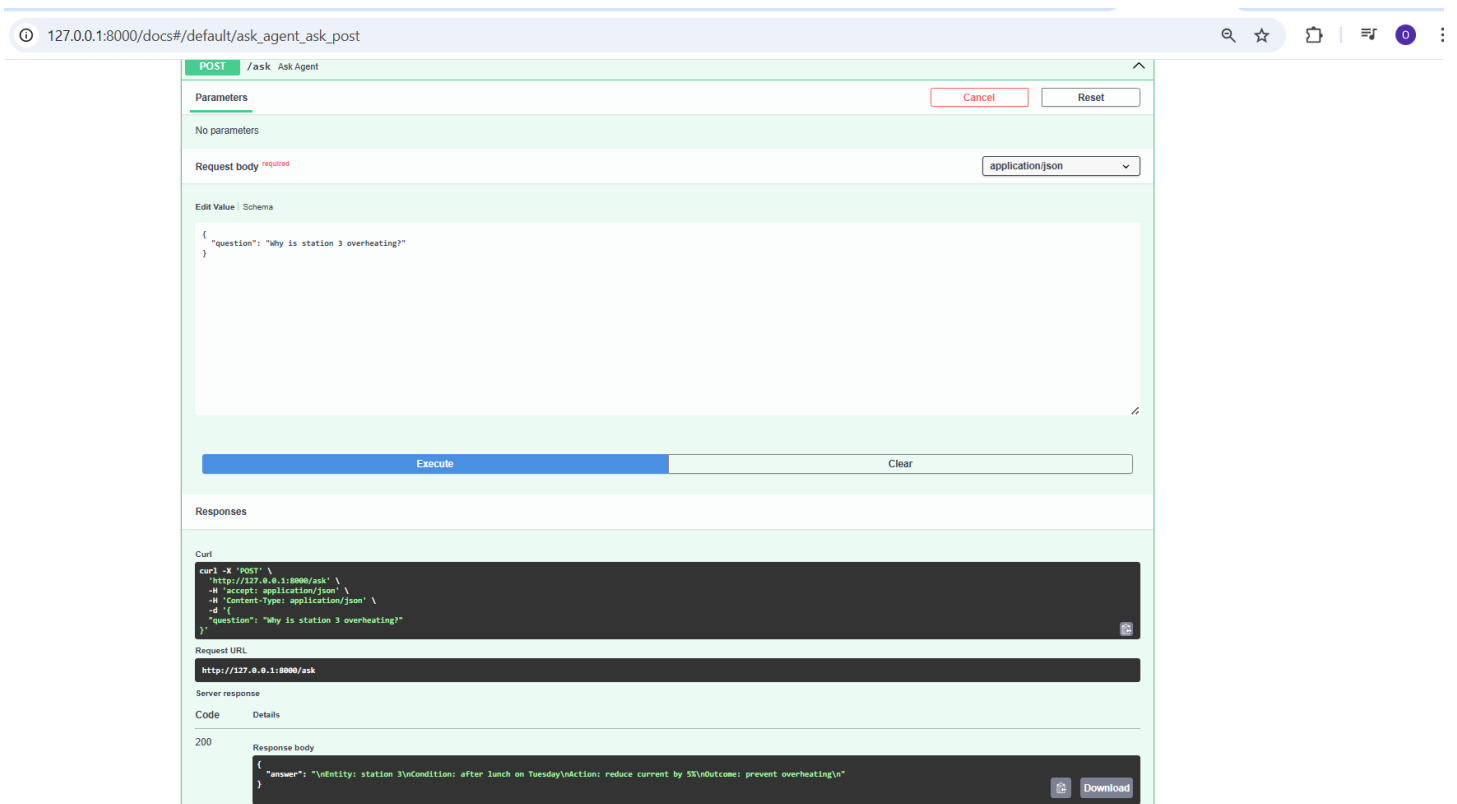
## Step 6 — Human Approval & Memory Replacement

### Step 6A — Ask Before Approval

#### API: POST /ask

- Input:

```
{
  "question": "Why is station 3 overheating?"
}
```
- Output:  
Entity: station 3  
Condition: after lunch on Tuesday  
Action: **reduce** current by 5%  
Outcome: prevent overheating



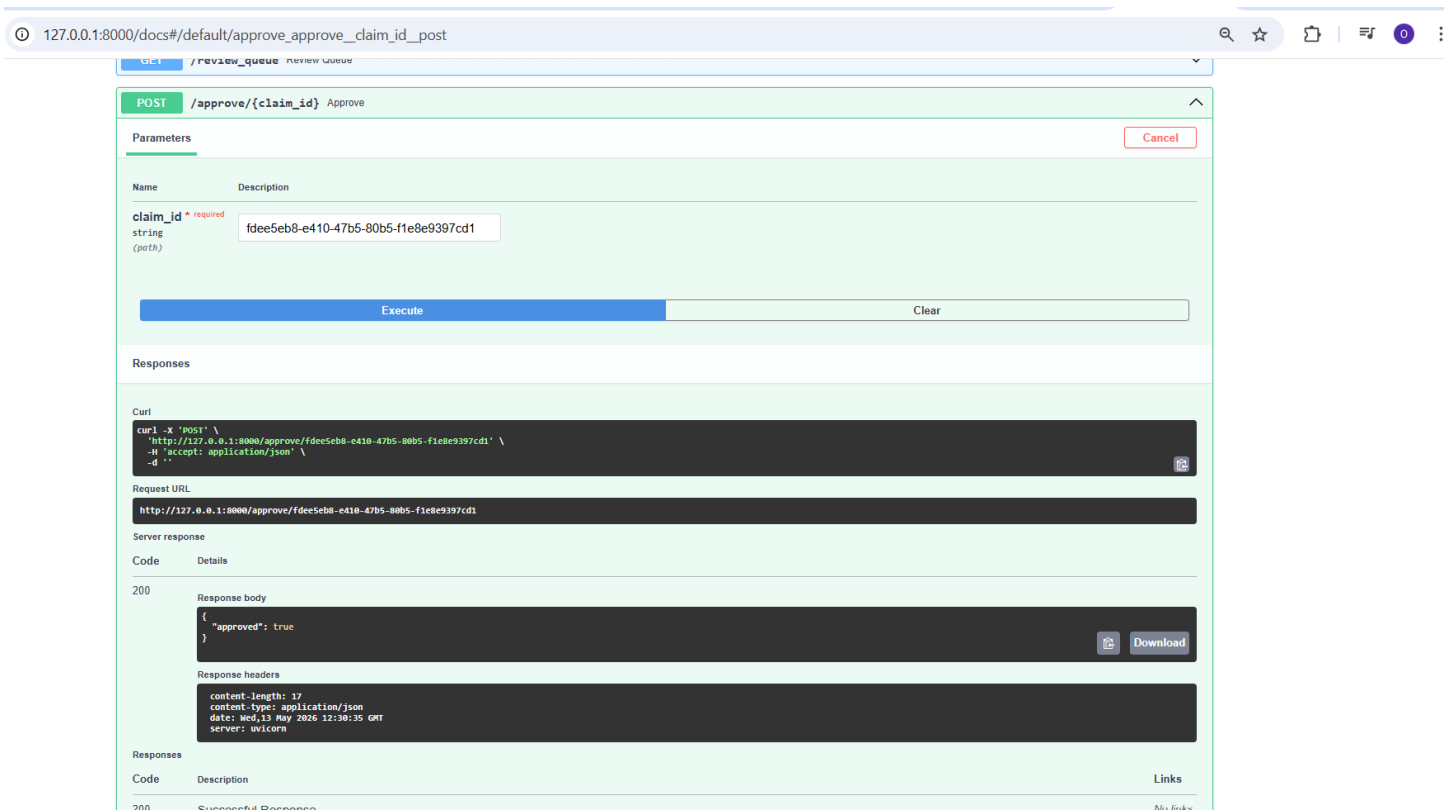
- Explain:
  - + The system still trusts the previously accepted knowledge
  - + The conflicting claim has not been approved yet
  - + The system avoids blindly overwriting memory
  - The system prioritizes verified operational memory over unverified conflicting claims.

## Step 6B — Human Approval

### API: POST /approve/{claim\_id}

- Input: output from step 4 ( get "claim\_id" from Review queue)
- Output:

```
{
  "approved": true
}
```



- Explain:
  - + Human supervisor approves the newer operational claim
  - + The system replaces outdated contradictory memory
  - + Operational memory is updated safely
  - The system supports human-governed continual learning.

## Step 6C — Ask Again After Approval

### API: POST /ask

- Input:
 

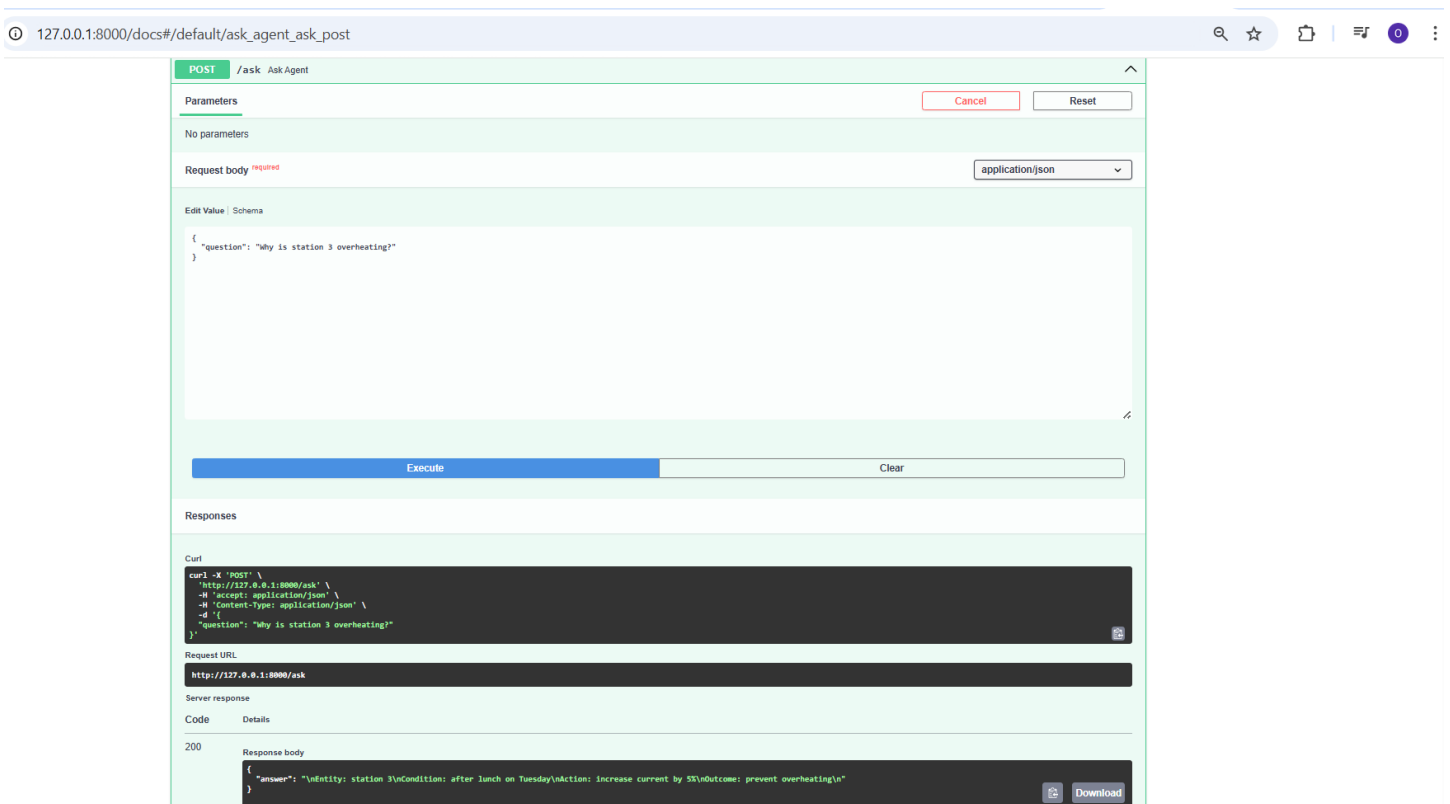
```
{
  "question": "Why is station 3 overheating?"
}
```
- Output:
 

Entity: station 3

Condition: after lunch on Tuesday

Action: **increase** current by 5%

Outcome: prevent overheating



- Explain:
  - + The system now retrieves the newly approved knowledge
  - + Old contradictory memory was replaced
  - + Operational truth evolved after human review
  - The system does not only store knowledge. It governs and updates operational memory safely.

## Step 7 — Metrics Demo

### API: [GET /metrics](#)

- Output:

```
{
  "accepted_claims": 2,
  "conflicts": 1,
  "quarantined": 1,
  "retrieval_hits": 2
}
```

127.0.0.1:8000/docs#/ POST /ask Ask Agent

GET /metrics Metrics

Parameters

No parameters

Execute Clear

Responses

Curl

```
curl -X 'GET' \
  'http://127.0.0.1:8000/metrics' \
  -H 'accept: application/json'
```

Request URL

http://127.0.0.1:8000/metrics

Server response

Code Details

200

Response body

```
{
  "accepted_claims": 2,
  "conflicts": 1,
  "quarantined": 1,
  "retrieval_hits": 2
}
```

Response headers

```
content-length: 70
content-type: application/json
date: Wed, 13 May 2020 12:32:58 GMT
server: nginx
```

- Explain: The system tracks measurable continual learning behavior instead of relying on subjective evaluation.